PCY

1. What is the PCY Algorithm?

The **PCY (Park-Chen-Yu) Algorithm** is an optimization of the standard Apriori algorithm designed for frequent itemset mining. It specifically addresses the **memory bottleneck** that occurs in Pass 2 of Apriori, where candidate generation (C_2) causes a massive explosion in the number of pairs that must be tracked in main memory.

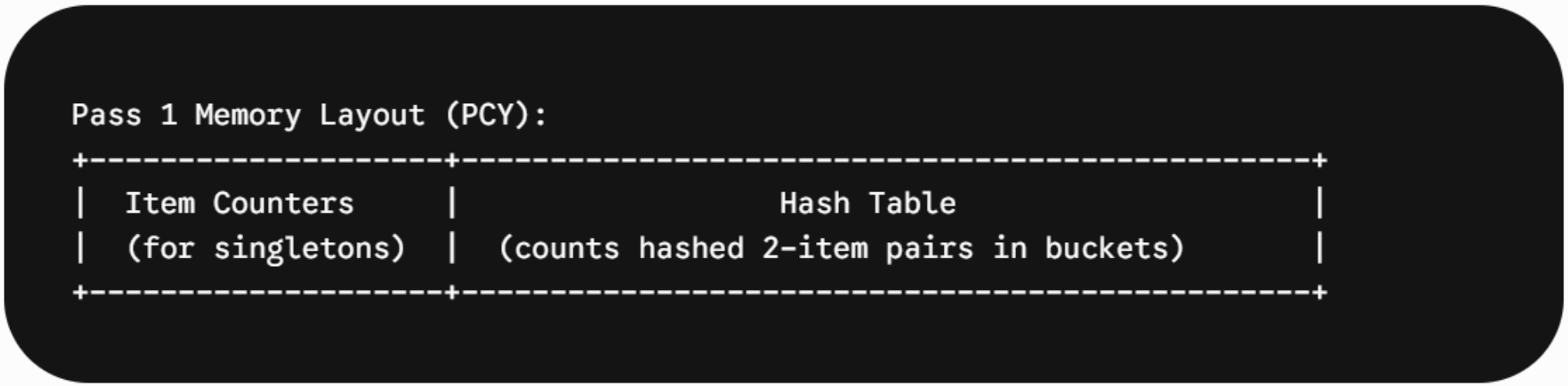
The Core Problem in Apriori

In standard Apriori:

1. **Pass 1:** Counts individual items to find frequent 1-itemsets (L_1). Memory usage is very low because N items require only N counters.
2. **Pass 2:** Generates candidate 2-itemsets (C_2) by taking all combinations of L_1 items ($L_1 \times L_1$). If L_1 has 10^5 items, C_2 will contain roughly $\frac{10^5 \times 10^5}{2} \approx 5 \times 10^9$ pairs. Allocating integer counters for billions of candidate pairs consumes gigabytes of RAM, often exceeding memory limits.

2. The PCY Solution: Utilizing Unused Pass 1 Memory

During **Pass 1** of Apriori, main memory is mostly empty because item counters take up very little space. PCY takes advantage of this free space by introducing a **Hash Table for 2-item pairs**.



3. How PCY Works: Step-by-Step

Pass 1: Counting Items & Hashing Pairs

For every transaction scanned in the dataset:

1. **Count Singletons:** Increment the counter for each individual item.
2. **Hash Pairs:** Generate all possible 2-item pairs in the transaction. Pass each pair $\{i, j\}$ through a hash function $h(i, j)$ that maps it to a bucket index in the hash table. Increment the counter for that bucket.

$$\text{Bucket Index} = h(i, j) = (\text{val}(i) \times \text{val}(j)) \pmod{k}$$

Between Pass 1 & Pass 2: The Bit Vector Trick

Once Pass 1 finishes:

1. Compare each bucket counter against the minimum support threshold (s).
2. Compress the array of bucket counters into a **Bit Vector (Bitmap)**:
    - If $\text{Bucket Count} \geq s$, set its bit to `1` (Frequent Bucket).
    - If $\text{Bucket Count} < s$, set its bit to `0` (Infrequent Bucket).
3. **Discard the integer hash table** and keep only the Bit Vector in memory for Pass 2.

Why a Bit Vector?
An integer counter takes 4 or 8 bytes. A bit takes $1/32^{\text{nd}}$ or $1/64^{\text{th}}$ of that space. A hash table of 100 million buckets compresses down to roughly **12.5 MB** of memory!

Pass 2: Candidate Generation (C_2) & Verification

When forming candidate pairs for C_2, PCY applies a strict **two-step filter**. A pair $\{i, j\}$ is admitted into C_2 **only if**:

1. **Singleton Condition:** Both item $i \in L_1$ and item $j \in L_1$.
2. **Bit Vector Condition:** The hash bucket bit for $\{i, j\}$ is `1` (i.e., $\text{Bit}[h(i, j)] = 1$).

If $h(i, j)$ maps to a bit of `0`, the pair $\{i, j\}$ is **immediately pruned** without allocating a counter for it. PCY then performs a second scan over the database to count actual occurrences of the surviving candidate pairs in C_2.

4. Performance Comparison: PCY vs. Apriori

Feature / Metric	Standard Apriori Algorithm	PCY Algorithm
Pass 1 Memory Footprint	Low (only stores singleton item counters).	Medium (stores singleton counters **+ Hash Table**).
Pass 2 Memory Footprint	**Very High** (allocates integer counters for all $L_1 \times L_1$ pairs).	**Low** (allocates integer counters **only** for pairs passing Bit Vector check).
Candidate C_2 Search Space	Unfiltered ($\|C_2\| = \frac{\|L_1\| \times (\|L_1\| - 1)}{2}$).	Pruned drastically (often reduced by 50%–90%).
Pass 2 Database Scan Time	Slow (must check and update counters for a large C_2).	Fast (checks occurrences for a significantly smaller C_2).
Key Limitation / Weakness	High memory overflow risk on large datasets.	**Hash Collisions**: Multiple infrequent pairs can hash to the same bucket and make it appear frequent.
Final Frequent Sets (L_k)	Identical results.	Identical results.

5. Understanding PCY Hash Collisions (False Positives)

Because the number of possible pairs is much larger than the hash table size (k), **collisions are inevitable**.

Example Scenario

Suppose minimum support threshold $s = 3$:

- Pair $\{a, b\}$ occurs 2 times.
- Pair $\{c, d\}$ occurs 1 time.

If both $\{a, b\}$ and $\{c, d\}$ hash to **Bucket 3**:

$$\text{Bucket 3 Count} = 2 + 1 = 3$$

Because $3 \geq s$, **Bucket 3 becomes marked as** `1` **(Frequent)** in the bit vector. As a result:

- Both $\{a, b\}$ and $\{c, d\}$ pass into C_2 as candidate pairs, even though neither pair is actually frequent on its own!

Takeaway: Hash collisions produce **False Candidates** (extra candidate pairs to count in Pass 2), but **NEVER False Negatives** (frequent pairs will never be pruned). Pass 2 verifies true counts, guaranteeing that the final output L_2 is 100% accurate.

Mathematical breakdown of the memory allocation for the Park-Chen-Yu (PCY) Algorithm across Pass 1 and Pass 2.

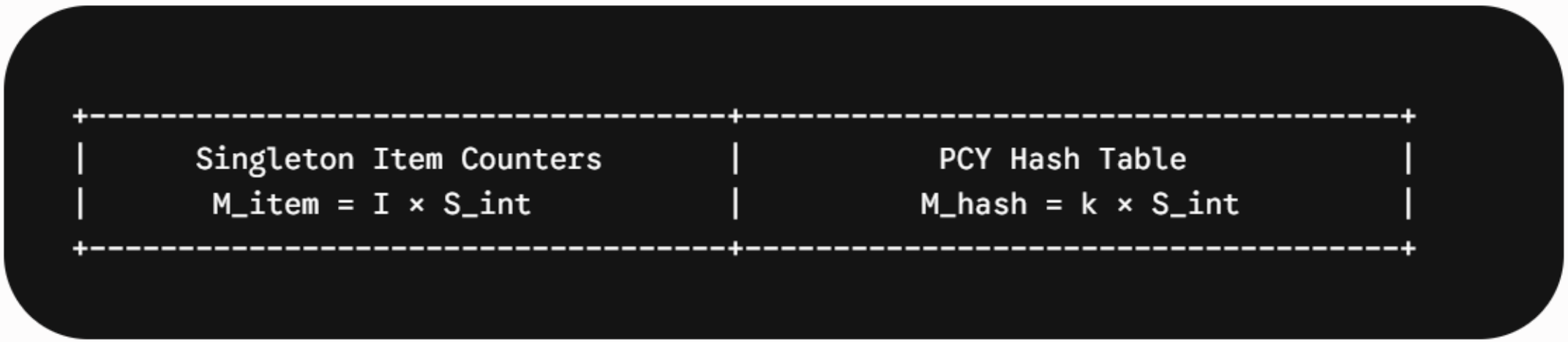
Key Variables & Notation

To formulate the equations, let:

- I: Total number of distinct items in the dataset.
- L_1: Number of frequent individual items (singletons) after Pass 1 ($L_1 \leq I$).
- k: Total number of buckets in the PCY hash table.
- C_2: Number of candidate 2-item pairs tracked in Pass 2.
- S_{int}: Bytes required for an integer counter (typically **4 bytes** for a 32-bit integer).
- S_{ptr}: Bytes required for a hash table/array pointer (typically **8 bytes** on a 64-bit system).

Pass 1: Memory Allocation Breakdown

During Pass 1, main memory is split into two primary components: **Singleton Item Counters** and the **PCY Hash Table**.



1. Singleton Counter Memory (M_{item})

Each distinct item needs an integer counter to keep track of its frequency.

$$M_{\text{item}} = I \times S_{\text{int}}$$

2. Hash Table Memory (M_{hash})

PCY fills the unused memory with a hash table consisting of k bucket counters.

$$M_{\text{hash}} = k \times S_{\text{int}}$$

Total Pass 1 Memory ($M_{\text{Pass 1}}$)

$$M_{\text{Pass 1}} = M_{\text{item}} + M_{\text{hash}} = (I \times S_{\text{int}}) + (k \times S_{\text{int}}) = (I + k) \times S_{\text{int}}$$

Example:
If $I = 100,000$ items, $k = 100,000,000$ buckets, and $S_{\text{int}} = 4$ bytes:

- $M_{\text{item}} = 100,000 \times 4\ \text{B} = 0.4\ \text{MB}$
- $M_{\text{hash}} = 100,000,000 \times 4\ \text{B} = 400\ \text{MB}$
- $M_{\text{Pass 1}} \approx 400.4\ \text{MB}$

Intermediate Step: Bit Vector Compression

Between Pass 1 and Pass 2, the integer hash table is evaluated against the support threshold s and converted into a **Bit Vector (Bitmap)**. The integer hash table is then freed from RAM.

Bit Vector Memory (M_{bit})

Since each bucket requires only **1 bit** instead of 4 bytes (32 bits):

$$M_{\text{bit}} = \left\lceil \frac{k}{8} \right\rceil \text{ bytes}$$

Compression Efficiency:
Switching from the integer hash table to the bit vector reduces its memory footprint by a factor of 32:

$$\frac{M_{\text{hash}}}{M_{\text{bit}}} = \frac{k \times 4 \text{ bytes}}{k/8 \text{ bytes}} = 32 \times \text{ reduction}$$

For $k = 100,000,000$, $M_{\text{bit}} = \frac{100,000,000}{8} \approx 12.5 \text{ MB}$ (down from 400 MB!).

Pass 2: Memory Allocation Breakdown

During Pass 2, memory is allocated for the **Bit Vector**, the **Frequent 1-itemsets (L_1)**, and the **Candidate Pair Counters (C_2)**.

Bit Vector	Frequent L_1	Candidate Pair Counters (C_2)
$M_{bit} = k / 8$	$M_{L1} = \|L1\| \times S_{int}$	$M_{C2} = \|C_2\| \times (2 \times S_{int} + S_{counter})$

1. Retained Bit Vector Memory (M_{bit})

$$M_{\text{bit}} = \frac{k}{8} \text{ bytes}$$

2. Frequent Items List (M_{L_1})

Only the items that met support threshold s are kept in L_1:

$$M_{L_1} = |L_1| \times S_{\text{int}}$$

3. Candidate 2-Itemset Counters (M_{C_2})

To count actual occurrences, each candidate pair in C_2 needs:

- Item IDs for the pair (2 integers = $2 \times S_{\text{int}}$).
- An occurrence counter (S_{int}).

$$M_{C_2} = |C_2| \times (3 \times S_{\text{int}})$$

How PCY Redefines $|C_2|$ Compared to Apriori:

- **Standard Apriori:** $|C_2| = \frac{|L_1| \times (|L_1| - 1)}{2}$
- **PCY Algorithm:** $|C_2| = \{\{i, j\} \in L_1 \times L_1 \mid \text{BitVector}[h(i, j)] = 1\}$

If the hash table successfully prunes $p\%$ of non-frequent pairs (where p is the filter efficiency, typically 50%–90%), then:

$$|C_2|_{\text{PCY}} \approx (1 - p) \times |C_2|_{\text{Apriori}}$$

Total Pass 2 Memory ($M_{\text{Pass 2}}$)

$$M_{\text{Pass 2}} = M_{\text{bit}} + M_{L_1} + M_{C_2}$$

$$M_{\text{Pass 2}} = \left(\frac{k}{8}\right) + \left(|L_1| \times S_{\text{int}}\right) + \left(|C_2|_{\text{PCY}} \times \mathbf{3} \cdot S_{\text{int}}\right)$$

Memory Comparison: PCY vs. Apriori

Phase	Standard Apriori Memory	PCY Algorithm Memory
Pass 1	$I \times S_{\text{int}}$	$(I + k) \times S_{\text{int}}$
Intermediate	—	Compresses k ints $\rightarrow \frac{k}{8}$ bytes
Pass 2	$(\lvert L_1 \rvert \times S_{\text{int}}) + \left(\frac{\lvert L_1 \rvert (\lvert L_1 \rvert - 1)}{2} \times 3 \cdot S_{\text{int}} \right)$	$\left(\frac{k}{8} \right) + (\lvert L_1 \rvert \times S_{\text{int}}) + (\lvert C_2 \rvert_{\text{PCY}} \times 3 \cdot S_{\text{int}})$

Summary of Trade-offs

1. **Pass 1:** PCY uses **more memory** than Apriori by adding $k \times S_{\text{int}}$ bytes for the hash table, taking advantage of main memory that would otherwise sit idle.
2. **Pass 2:** PCY uses **dramatically less memory** than Apriori because $|C_2|_{\text{PCY}} \ll |C_2|_{\text{Apriori}}$. The small overhead of keeping the Bit Vector ($\frac{k}{8}$ bytes) is far outweighed by the memory saved from pruning millions of candidate counters.

Problem Statement

Given a transactional database with 5 transactions and 5 items, find all frequent 2-itemsets using the PCY Algorithm.

- **Transactions (T):**
  - $T_1 = \{1, 2, 3\}$
  - $T_2 = \{1, 2, 4\}$
  - $T_3 = \{1, 3, 4\}$
  - $T_4 = \{2, 3, 4\}$
  - $T_5 = \{2, 3, 5\}$
- **Minimum Support Threshold (s):** 3 transactions (or 60%)
- **Hash Table Size (k):** 7 buckets (indexed 0 to 6)
- **Hash Function:** $h(i, j) = (i \times j) \pmod 7$

Pass 1: Item Counting & Hash Table Construction

During **Pass 1**, we perform two actions simultaneously for every transaction:

1. Count the occurrences of individual items.
2. Generate all 2-item pairs in the transaction, hash each pair, and increment the count of the corresponding bucket in memory.

Step 1.1: Count Individual Items

Item	Transactions Present	Count	Frequent? (≥ 3)
1	T_1, T_2, T_3	3	Yes
2	T_1, T_2, T_4, T_5	4	Yes
3	T_1, T_3, T_4, T_5	4	Yes
4	T_2, T_3, T_4	3	Yes
5	T_5	1	No

Frequent 1-Itemsets (L_1): $\{1, 2, 3, 4\}$

Step 1.2: Hash Pairs to Buckets

Let's break down all pairs present in each transaction and compute $h(i, j) = (i \times j) \pmod 7$:

- $T_1 = \{1, 2, 3\}$
  - $\{1, 2\} \implies (1 \times 2) \pmod 7 = 2$
  - $\{1, 3\} \implies (1 \times 3) \pmod 7 = 3$
  - $\{2, 3\} \implies (2 \times 3) \pmod 7 = 6$
- $T_2 = \{1, 2, 4\}$
  - $\{1, 2\} \implies (1 \times 2) \pmod 7 = 2$
  - $\{1, 4\} \implies (1 \times 4) \pmod 7 = 4$
  - $\{2, 4\} \implies (2 \times 4) \pmod 7 = 1$

- $T_3 = \{1, 3, 4\}$
  - $\{1, 3\} \implies (1 \times 3) \pmod 7 = 3$
  - $\{1, 4\} \implies (1 \times 4) \pmod 7 = 4$
  - $\{3, 4\} \implies (3 \times 4) \pmod 7 = 5$
- $T_4 = \{2, 3, 4\}$
  - $\{2, 3\} \implies (2 \times 3) \pmod 7 = 6$
  - $\{2, 4\} \implies (2 \times 4) \pmod 7 = 1$
  - $\{3, 4\} \implies (3 \times 4) \pmod 7 = 5$
- $T_5 = \{2, 3, 5\}$
  - $\{2, 3\} \implies (2 \times 3) \pmod 7 = 6$
  - $\{2, 5\} \implies (2 \times 5) \pmod 7 = 3$
  - $\{3, 5\} \implies (3 \times 5) \pmod 7 = 1$

Step 1.3: Aggregate Bucket Counts

Bucket Index	Pairs Hashed to Bucket	Total Bucket Count
0	None	0
1	$\{2, 4\}$ (2 times) + $\{3, 5\}$ (1 time)	3
2	$\{1, 2\}$ (2 times)	2
3	$\{1, 3\}$ (2 times) + $\{2, 5\}$ (1 time)	3
4	$\{1, 4\}$ (2 times)	2
5	$\{3, 4\}$ (2 times)	2
6	$\{2, 3\}$ (3 times)	3

Between Pass 1 & Pass 2: Bit Vector (Bitmap) Creation

Convert the integer array of bucket counts into a compact **Bit Vector**. If a bucket count $\geq s = 3$, set its bit to `1` (frequent bucket); otherwise, set it to `0`.

$$\text{Bit Vector} = [0, 1, 0, 1, 0, 0, 1]$$

- **Frequent Buckets (Bit = 1):** Bucket 1, Bucket 3, Bucket 6.
- **Infrequent Buckets (Bit = 0):** Bucket 0, Bucket 2, Bucket 4, Bucket 5.

Pass 2: Candidate Generation (C_2) & Filtering

In standard **Apriori**, C_2 includes every pair formed by L_1:

$$\text{Apriori } C_2 = \{\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,4\},\{3,4\}\} \quad (6 \text{ pairs})$$

In **PCY**, a pair $\{i, j\}$ becomes a candidate in C_2 **only if**:

1. Both items $i \in L_1$ and $j \in L_1$.
2. The pair hashes to a bucket whose bit vector value is **1**.

Let's test each pair from L_1:

1. $\{1,2\}: h(1,2) = 2 \implies \text{Bit}[2] = 0 \implies$ **PRUNED**
2. $\{1,3\}: h(1,3) = 3 \implies \text{Bit}[3] = 1 \implies$ **CANDIDATE**
3. $\{1,4\}: h(1,4) = 4 \implies \text{Bit}[4] = 0 \implies$ **PRUNED**
4. $\{2,3\}: h(2,3) = 6 \implies \text{Bit}[6] = 1 \implies$ **CANDIDATE**
5. $\{2,4\}: h(2,4) = 1 \implies \text{Bit}[1] = 1 \implies$ **CANDIDATE**
6. $\{3,4\}: h(3,4) = 5 \implies \text{Bit}[5] = 0 \implies$ **PRUNED**

PCY Result: $C_2 = \{\{1,3\}, \{2,3\}, \{2,4\}\}$

(Candidate count reduced from 6 to 3!)

Pass 2: Counting Candidates & Finding L_2

Scan the dataset one last time to count actual occurrences of the candidate pairs in C_2:

Candidate Pair	Transactions Present	Actual Count	Frequent? (≥ 3)
$\{1, 3\}$	T_1, T_3	**2**	No
$\{2, 3\}$	T_1, T_4, T_5	**3**	**Yes**
$\{2, 4\}$	T_2, T_4	**2**	No

Final Output

- **Frequent 1-Itemsets (L_1):** $\{\{1\}, \{2\}, \{3\}, \{4\}\}$
- **Frequent 2-Itemsets (L_2):** $\{\{2, 3\}\}$

Key Takeaways

- **Memory Savings:** By storing a 1-bit per bucket bitmap instead of 4-byte/8-byte counters during Pass 2, PCY frees up memory to handle massive candidate search spaces.
- **False Positives:** A bucket can be frequent due to multiple infrequent pairs hashing to it (e.g., $\{2, 4\}$ with count 2 and $\{3, 5\}$ with count 1 combined to make Bucket 1 frequent with count 3). Pass 2 verifies the actual count to eliminate false positives.